

Codex + Claude Code

The Concepts Companion

Everything the video had room for, plus the parts it didn't.

by Mark Kashef
Early AI Dopters

How to use this companion

The Field Guide replays the video. This Companion goes deeper. Glossary so no term trips you up. A real cost calculator so you know what to budget. Advanced flags the video didn't have time to cover. Configuration files for when you're ready to dial things in. A list of the most common mistakes I see people make with the duo. And a section on when you should NOT use it. Read it once, skim it for reference forever.

Contents

- 01** Glossary
- 02** The Duo Philosophy
- 03** Tokenomics: What This Costs
- 04** Advanced Flags Reference
- 05** Configuration Files
- 06** Common Mistakes And Gotchas
- 07** When NOT To Use The Duo
- 08** Troubleshooting
- 09** Reading List + What's Next

01

Glossary

Every term the video used, plain English.

If a word in the video sounded vague or technical, it should be in here. Bookmark this section and come back to it any time something doesn't click.

AI Coding Assistant

An AI tool that helps you write, review, or debug code. Claude Code and Codex are both AI coding assistants. Different models, different strengths.

API Key

A long string of characters that lets a program access a paid service on your behalf. The Codex plugin does NOT need a separate API key — it uses your existing ChatGPT login.

Background Job

A task that runs in a separate process so you don't have to wait for it. When you fire `/codex:review --background`, Codex starts reviewing in the background and you keep working with Claude in the foreground.

CLI

Short for Command-Line Interface. The dark terminal screen where developers type commands. The Codex CLI is a program that runs on your computer; the Codex plugin connects Claude Code to it.

Codex

OpenAI's coding-focused AI model and CLI tool. Available with a ChatGPT Plus subscription. The plugin lets you call Codex from inside Claude Code without switching tools.

Codex Plugin (codex-plugin-cc)

An official OpenAI plugin for Claude Code. Adds `/codex:*` slash commands so you can call Codex without leaving the Claude Code window. Available in the Claude Code plugin marketplace.

Diff

Short for 'difference.' The set of changes you've made compared to a previous saved state. When Codex reviews, it reviews the diff — only the new work, not your whole codebase.

Foreground vs Background

Foreground = Claude waits for the result. You stare at a spinner. Fine for tiny changes. Background = Claude doesn't wait. You keep working. Better for non-trivial reviews. The flag `--background` flips a command into background mode.

Git

A version control system. Tracks every change you make to your code over time. The Codex plugin uses git under the hood to figure out what's changed (the diff).

Hook

A built-in feature in Claude Code that fires automatically at specific moments. Like an event listener. The review gate uses a Stop hook, which fires when Claude tries to finish a turn.

OWASP Top Ten

A famous list maintained by a security organization that names the ten most common categories of web app security problems. `/codex:review` checks for these among other things.

Pagination

Splitting a list of results across pages. If you have 100,000 links and ask the database for all of them at once, the page melts. Pagination breaks them into batches of, say, 20 at a time.

Plugin Marketplace

A directory inside Claude Code where you can install community plugins. `/plugin marketplace add` is how you register a plugin's source; `/plugin install` is how you actually install it.

Race Condition

When two things happen at the exact same instant and step on each other. The Linkboard click counter has a race condition: two clicks at once can cause one to be lost. Hard to reproduce, easy to ship. Codex catches them.

Read-Only Command

A command that looks at code but does NOT edit it. `/codex:review` and `/codex:adversarial-review` are both read-only. They produce findings, not changes.

Slash Command

A command in Claude Code that starts with a forward slash, like `/codex:review`. The plugin adds a set of `/codex:*` slash commands to your Claude Code session.

Slug

The short identifier in a URL. In `bit.ly/abc123`, 'abc123' is the slug. Linkboard generates random slugs for shortened links.

Soft Delete

Instead of erasing data, you mark it as deleted but keep the data around. Lets you recover if someone hits the delete button by mistake. The opposite is a hard delete or force delete, which actually removes the data.

Steerable

A command you can guide with focus text. `/codex:adversarial-review` is steerable: you add focus text after the flags and Codex grills exactly that concern. `/codex:review` is NOT steerable — it always does the full pass and ignores focus text.

Stop Hook

A specific kind of hook that fires when Claude tries to finish a turn. The review gate uses a Stop hook to intercept Claude's exit and run a Codex review on the way out.

Tokens

The currency AI models bill in. Every prompt and response costs tokens. The review gate is expensive because it adds a Codex review on every Claude finish — five back-and-forths in a session = five extra reviews = lots of tokens.

Working Tree

Git terminology for the files you currently have on disk, including any uncommitted changes. When Codex reviews 'the working tree,' it's reviewing your latest changes.

--background flag

Added at the end of a Codex command to make it run in a background process. Pair with /codex:status (check progress) and /codex:result (read the output).

--enable-review-gate

A flag for /codex:setup that turns on the auto-review-on-finish behavior. Pair with --disable-review-gate to turn it off.

--effort

Tells Codex how hard to think. Values: none, minimal, low, medium, high, xhigh. Default is medium. Use xhigh for very high-stakes reviews, but it costs more tokens and takes longer.

--model

Lets you pick which Codex model to use. Default is fine for most things. 'spark' maps to a faster, cheaper model good for quick passes.

--resume / --fresh

For /codex:rescue, these decide whether to continue a previous Codex thread or start a new one. Default behavior asks you. Pass --resume to continue or --fresh to start over without asking.

The Duo Philosophy

Why both, and what each one is actually good at.

There's a deeper pattern under the Claude vs Codex debate that's worth naming. The two models are trained on different priorities and they produce different output styles. Pretending they're interchangeable is what makes the choice feel hard. Understanding what each one is actually optimized for makes the choice obvious — you use both.

Claude Code is great at:

- **Planning architecture.** What modules you need, how they fit together, what comes first.
- **Writing copy.** Microcopy, error messages, page headlines, README files. Voice and warmth.
- **Design taste.** What looks good, what feels intuitive, what UI patterns to reach for.
- **Big-picture thinking.** What direction the product should go, what features matter most.
- **Pair-programming feel.** Working with you turn by turn, holding context across long sessions.

Codex is great at:

- **Code audits.** Spotting bugs that have been sitting in your code for weeks unnoticed.
- **Edge cases.** What happens when the input is empty, malformed, or arrives twice in a row.
- **Race conditions.** What breaks when two things happen at the same time.
- **Security.** The OWASP top ten, injection attacks, validation gaps, auth holes.
- **Long surgical refactors.** Methodical, focused, won't drift off course.
- **Adversarial review.** Playing devil's advocate on a design before it ships.

What neither one is great at right now:

- **Original product strategy.** Both can help refine your strategy, but you still need to bring the judgment.
- **Knowing when to stop.** Both will keep going until you say done. The discipline is yours.
- **Knowing when they're wrong.** Both can confidently produce bad output. Always verify, especially for high-stakes work.

The teaching line

Claude plans, designs, writes copy. Codex audits and does surgery. Together they're a duo. The right fight isn't picking which is better. The right fight is using both for what each one is genuinely good at.

Tokenomics

What this actually costs, broken down.

The video says \$120/month covers the duo. Here's the actual breakdown so you know what you're paying for and where the cost spikes hide.

Base subscriptions:

- **Claude Max** — \$100/month. Gets you Claude Code with high usage limits. The 'enough' tier for most independent builders.
- **ChatGPT Plus** — \$20/month. Gets you Codex (plus everything else ChatGPT offers).
- **Total: \$120/month.** Both are consumer subscriptions. No enterprise contract. No API key billing surprises.

Cost spikes to watch:

- **Review gate left on overnight.** Every Claude finish triggers another Codex review. Five back-and-forths a session = five extra reviews. A long session can run through a LOT of tokens. Always `/codex:setup --disable-review-gate` when you're done.
- **Adversarial review with `--effort xhigh`.** Higher effort = more thinking = more tokens per review. Use it for genuinely high-stakes changes, not casual reviews.
- **Codex rescue without scope.** If the rescue task is open-ended ('investigate why our performance is bad'), Codex can spend a lot of compute. Tighten the task with concrete constraints when possible.

How to keep costs predictable:

- Use `/codex:review` (one shot) for most reviews. Only use the gate for focused sessions.
- Default to `--effort medium` (the unspoken default). Reach for `xhigh` only when warranted.
- Pair `--background` with `/codex:status` — don't accidentally run multiple reviews in foreground when you meant to start one in background.
- Set a project-level config in **`.codex/config.toml`** with a smaller default model if you want consistent cheaper reviews. See the next section.

Advanced Flags Reference

The flags the video didn't have time to cover.

--effort <level>

Tells Codex how hard to think. Values: none, minimal, low, medium, high, xhigh. Default is medium. Use xhigh for very high-stakes reviews where you want maximum scrutiny. Costs more tokens and takes longer.

```
/codex:rescue --effort xhigh investigate the regression deeply
```

--model <model>

Picks which Codex model to use. Most people stick with the default. The 'spark' shorthand maps to a faster, cheaper model good for quick passes when speed matters more than depth.

```
/codex:rescue --model spark fix the flaky test quickly
```

--resume / --fresh

For /codex:rescue. Decides whether to continue a previous Codex thread or start over. The default behavior asks you. Pass --resume to continue or --fresh to start fresh without being asked.

```
/codex:rescue --resume apply the top fix from the last run
```

--base <branch>

For /codex:review and /codex:adversarial-review. Tells Codex to compare your current branch against a different one (typically main) instead of just looking at uncommitted changes. Useful for reviewing a whole feature branch before opening a PR.

```
/codex:review --base main --background
```

--wait

Forces a command into foreground mode (the opposite of --background). Useful when you explicitly want to wait for the result before doing anything else.

```
/codex:review --wait
```


Combining flags

Most flags can combine. Common combos: `--background --effort xhigh` for a thorough background review. `--resume --model spark` for quickly applying the previous fix.

```
/codex:adversarial-review --base main --background challenge auth flow  
security
```

05

Configuration Files

Set defaults so you don't have to type the same flags every time.

If you find yourself always passing `--effort high` or `--model spark`, save them as defaults. The Codex plugin reads two configuration files in order:

- `~/.codex/config.toml` — user-level. Applies everywhere. Set the defaults you always want.
- `.codex/config.toml` in your project root — project-level. Overrides the user-level settings, but only for that project. Note: project configs only load if the project is marked trusted in your Codex settings.

Example config:

```
# ~/.codex/config.toml
```

```
model = "gpt-5.4-mini"
```

```
model_reasoning_effort = "high"
```

After saving this file, every Codex command in every project uses these defaults unless you pass different flags inline.

Why this matters

If you pin a project to `gpt-5.4-mini` at high effort, you get consistent, predictable reviews without having to remember the flags. Important for shared team setups and for keeping costs predictable across long sessions.

Heads up on the review gate

The review gate is registered as a Stop hook in Claude Code, NOT in the Codex config file. To toggle it, you use `/codex:setup --enable-review-gate` or `--disable-review-gate`, not the config file. Don't try to add `review_gate = true` to your config — it won't do anything.

Common Mistakes And Gotchas

The traps everyone falls into at least once.

1. Running `/codex:review` on your whole monorepo by accident

If your demo project doesn't have its own `.git` folder, git falls back to the nearest parent `.git`, which might be a much bigger repo. Codex then 'reviews' everything in that bigger repo. Fix: always run **git init** inside the project folder you actually want reviewed, then make at least one commit so there's a baseline.

2. Forgetting to disable the review gate

Easy to flip on, easy to forget about. Every Claude finish triggers another Codex review. Tokens vanish fast. Always run **`/codex:setup --disable-review-gate`** when you're done with a focused session.

3. Adding focus text to `/codex:review`

`/codex:review` is NOT steerable. It ignores anything you type after the command. If you want focus text, use `/codex:adversarial-review` instead.

4. Using `/codex:rescue` when you wanted a read-only opinion

`/codex:rescue` is write-capable by default. It can edit your code. If you want Codex to look at something specific without making any changes, use `/codex:adversarial-review` with focus text, OR tell rescue explicitly: 'do not change anything yet, just diagnose.'

5. Firing `--background` and forgetting to check the result

Background jobs don't auto-display when they finish. You have to explicitly run `/codex:status` to see if a job is done, then `/codex:result` to read the findings. Pair them every time.

6. Letting Codex run a 'review' on a brand-new empty project

If there's no diff (nothing has changed since the last commit), there's nothing to review. Codex will tell you so, but it still costs you a roundtrip. Make sure you have actual changes in the working tree before firing a review.

7. Asking Codex to plan

`/codex:adversarial-review` reviews CODE diffs, not free-form text plans. If you want Codex to review a plan, write the plan into a file (`PLAN.md`), commit it, and the review will pick up the new file. Or use `/codex:rescue` with explicit task text instead.

When NOT To Use The Duo

Times when one model alone is the better choice.

The duo is powerful, but it's not always worth the overhead. Here are the cases where pulling out Codex is actively the wrong move.

- **Throwaway prototypes.** If you're spiking on something for 20 minutes to see if an idea is worth pursuing, just let Claude write it. Adding a Codex review pass is overhead that slows you down for no real benefit.
- **Style nits and formatting.** Both models will happily debate formatting forever. If your linter and prettier are doing their job, Codex doesn't need to weigh in.
- **Pure copy work.** Microcopy, page text, button labels — this is Claude's home turf. Codex doesn't have a strong design or copy sense and its reviews will read like a code review of a poem.
- **UI design exploration.** When you're sketching out what an interface should look like, the feedback you want is taste-based, not safety-based. Stay with Claude.
- **Tiny one-line fixes.** If Claude added a missing semicolon or fixed a typo, running a full review pass is overkill. The duo earns its keep on non-trivial changes.
- **When you're in flow on creative work.** The whole point of the duo is to give you safety without breaking flow. If pulling in Codex IS what's breaking your flow, save it for the next checkpoint.

Rule of thumb

If the change is small and reversible, just ship it. If the change is non-trivial and a bug would hurt — users, money, trust, security — that's when the duo earns its keep.

Troubleshooting

What to do when something goes wrong.

Codex says it found nothing material

First, double-check there's actually a diff in the working tree (run **git status**). If there's no diff, there's nothing to review. If there IS a diff and Codex still finds nothing, that might just mean the code is solid — or you might want to try `/codex:adversarial-review` with specific focus text to push harder.

/codex:setup says Codex isn't installed

Run **npm install -g @openai/codex**, then run `/codex:setup` again. If you don't have Node.js installed at all, you'll need to install Node 18.18 or later first.

/codex:setup says Codex isn't authenticated

Run **!codex login** inside Claude Code. The exclamation point tells Claude Code to run a shell command. It opens a browser to sign in with your ChatGPT account.

The review is reviewing way more code than I expected

Your project might be using a parent folder's git repo as its working tree. Check with **git rev-parse --show-toplevel** from inside your project. If it points to a parent folder, run **git init** in your actual project folder and make a baseline commit.

The review gate is taking forever

The Stop hook has a 900-second timeout (15 minutes). If your reviews are taking that long, consider reducing `--effort`, using the dash dash background flag for the underlying review, or scoping the change smaller. Long-running gate reviews drain tokens AND your patience.

I want to see what Codex actually ran

After `/codex:result`, you'll see a Codex session ID. You can run **codex resume <session-id>** in a regular terminal to reopen that exact session in pure Codex CLI and inspect everything it did.

I want to fully reset and start over

Run `/codex:cancel` for any active jobs. Run `/codex:setup --disable-review-gate` to make sure the gate is off. Then exit Claude Code and reopen. Fresh slate.

Reading List + What's Next

Where to learn more.

Official references:

- **The Codex plugin source code on GitHub.** Search for `openai/codex-plugin-cc`. The README is the most accurate reference for every flag, every command, and every behavior. Updated as the plugin evolves.
- **The Codex CLI docs.** `developers.openai.com/codex`. Configuration, advanced features, and the actual CLI behavior the plugin wraps.
- **The Claude Code documentation.** Anthropic's docs cover Claude Code itself — plugins, hooks, slash commands, and the agent system.

Patterns the community is converging on:

- **Stop hook plugins.** The review gate uses one Stop hook. The community is building plugins that use the same mechanism for autonomous loops, multi-agent reviews, and conditional reviewers. If you want to build your own, study how the official Codex plugin's hook is structured and adapt.
- **Codex multi-agent mode.** Codex supports parallel sub-agents via `[features] multi_agent = true` in your config. Useful for large reviews where you want diff review and architecture review and UX review running in parallel.
- **Adversarial planning loops.** The Pattern 2 workflow (plan, adversarial review, reflect, revise) is the most underused move in modern AI-assisted coding. Practice it on every non-trivial feature.

What's coming next on the channel:

A follow-up video on building your own version of an autonomous Codex loop using a Stop hook. Same Linkboard demo, deeper dive on the architecture. If you want to be first when it drops, join Early AI Dopters — the link is in the video description.

If you got value from this guide

Drop DUO in the comments of the video to grab the full kit (this guide, the Field Guide, demo prompts, the install commands). And tell me which pattern you tried first — I read every reply.